

Programming with Java

Lab instructions for Practise Stage live event: Day 1 - 6

Contents

Lab 1: Introduction to Java and Eclipse 3
Lab 2: Java – The Basics 5
Lab 3: Introduction to Methods 8
Lab 4: Java – Getting Started with Conditionals 11
Lab 5: Java – More Conditionals 13
Lab 6: Java – Loops 15
Lab 7: Java – Arrays 17
Lab 8: Types I – Getting started 19
Lab 9: Types II – Fields and Methods 22
Lab 10: Enums and Strings 26
Lab 11: Collections and Generics 28
Lab 12: Java - Working with Files 30
Lab 13: Exceptions 34
Lab 14: Inheritance – Getting started 36
Lab 15: Inheritance – towards polymorphism 38
Lab 16: Pair Programming 42
Lab 17: Unit Testing 44
Lab 18: JDBC 49

Lab 1: Introduction to Java and Eclipse

Objective

In this lab, you will:

- create a Console program using the Eclipse IDE.
- run the program.

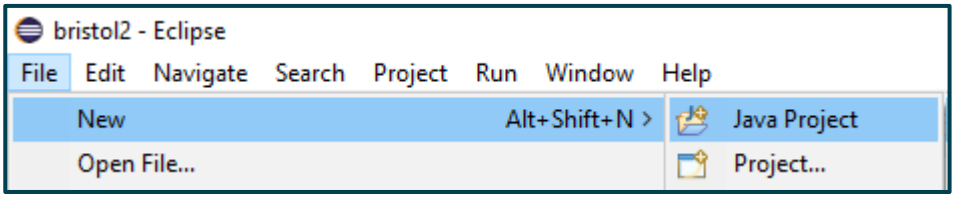
Overview

Eclipse is the development tool used by many Java developers and throughout this course. This lab will show you how to create and run a Java program.

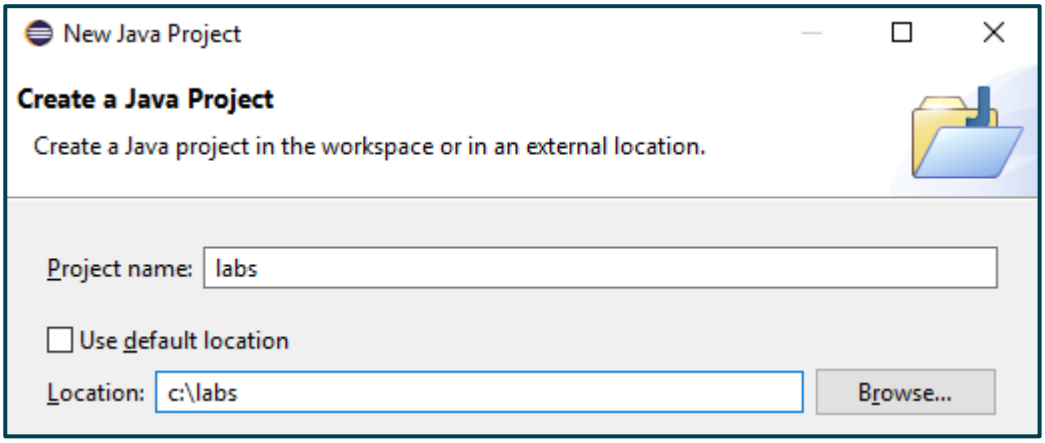
Creating a new Java application

Step by step

1. Launch Eclipse and type C:\Labs as the location for your 'workspace'.
2. Dismiss the Welcome window.
3. Choose menu option File > New > Java Project



4. Type the following project name and location and then accept the other default settings:



5. Click 'Finish'
6. Right click on the lab1 folder in your project explorer and then select

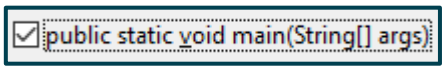
   New > Package

   name this package **lab01**

7. Right click on the **lab01** node and then select New > Class

   Name this class **Program**

   And make sure you select 

Part 2 - Writing the Code

Step by step

8. In the Program class's main method, type the following:

```
public static void main(String[] args) {
System.out.println("Hello, world!");     }
```

9. In the line below the System.out.println, type 'sys' and hit **Ctrl-Space** you'll find that *Content Assist* pops up a list of candidates.

   Choose the entry **syso** 'and then hit **Ctrl-Space.** The first entry in the context menu will type `System.out.println`.

   Inside the parentheses type the String "My name is --" fill the -- with your name

   Make sure you make full use of this time saving editor command.

Part 3 - Building and running the program

Step by step

10. Compiling the program is done by saving your code (Ctrl-s)
11. Press **Ctrl+F11** to run the program or click the green Run icon.

    A Console window will become active at the bottom of Eclipse.
    Please view the output for the *println* messages.

** End **

Lab 2: Java – The Basics

Objective

In this lab, you will:

- declare and initialise variables.
- practice using mathematical operators.

Part 1 – Declare and initialise local variables

Step by step

1. Back in the labs project you created in Lab1, create a new package called **lab02**.
2. Add a new class called **Program** to the *lab02* package with a main() method.

   Please refer to lab1's instructions if you need help.
3. Declare and initialise variables to hold your details such as:
   a. age (int)
   b. name (String)
   c. house number (int), street (String) and post code (String)
   d. telephone number (String)
   e. company you work for (String)
   f. salary (double)
   g. if you have a driving licence (boolean)
4. Use a println (or a series of println methods) to display the above information.

   You can put variable together inside a println() method using the + operator.



Part 2 – Doing some maths work

Step by step

5. Expand the **main** method.
6. Comment all the code you wrote in Part 1.
   Tip: Highlight the code and press **Ctrl-/**
7. Copy and paste the code below and carry out the tasks.
   Please try one task at a time, save and run your code to test your code at every step.

```
System.out.println("Initial Value: " + number);
int number = 5;

// Task 1
// - double it using the '*' operator
// - now double it again using the '*=' operator

System.out.println("\n1. After doubling it twice: " + number);

// Task 2
// - add 3 to it using the '+' operator
// - now add 3 to it using the '+=' operator

System.out.println("\n2. After adding 3 twice: " + number);

// Task 3 - subtract 12 from it using an appropriate 'compound' operator

System.out.println("\n3. After subtracting 12: " + number);

// Task 4 - divide the number (ought to be 14 now) by 3
// what do you think the answer is

System.out.println("\n4. After dividing by 3: " + number);

// Task 5 write 4 different statements that all do the same thing
// namely 'add 1 to the number'

System.out.println("\n5. After adding 1 four times: " + number);

// Task 6 decrement by 1 the value of number

System.out.println("\n6. After decrementing once: " + number);

// Task 7 put the remainder when dividing by 3 into 'remainder'

int remainder = 0;

System.out.println("\n7. Remainder when dividing by 3 is :" + remainder);

// Task 8
// decide what it will print before uncommenting the println()

int a = 2, b = 3, c = 5;
double d1, d2, d3, d4;


d1 = a + b * c / 2;
d2 = (a + b * c) / 2;
d3 = (a + b) * c / 2;
```



```
d4 = (a + b) * (c / 2);
// System.out.println("\n8. Values: " + d1 + " : " + d2 + " : " + d3
// + " : " + d4);

// Type your answer here-->

// Task 9

int p, q;
p = 10;
q = 10;
p += q++;

// Decide what the next line will print
// System.out.println("\n9. Result is: " + (p + q));
// Answer-->

// Task 10 – Uncomment the code below
//System.out.println("\n11.");

// Decide what the following 4 lines will print
// The 4th one might surprise you

// System.out.println("fred" + 3 + 4); // Answer-->
// System.out.println(3 + 4 + "fred"); // Answer-->
// System.out.println("" + 3 + 4); // Answer-->
// System.out.println(3 + ' ' + 4); // Answer-->
```

** End **

Lab 3: Introduction to Methods

Objective

The objectives of this practical session are as follows.

- To be able to write and invoke methods that have a varying number of parameters, some of which return a value.
- To accept user input in response to a prompt and process that data further including converting it to a different type of data.
- You'll also create and use a new class

Part 1 – Authoring a helper method

Step by step

1. Back in the labs project which you created in Lab1, add a new package called **lab03**.

   Please refer to lab1's instructions if you need help.

2. Add a new class called **Program** to the *lab03* package with a *main()* method.

3. Add a new method as **public static int getInt**(**String** prompt)

   This method has a String parameter called *prompt*, which it displays before getting an integer input from the user. It then returns an **int**.

   To get keyboard input (***System.in*** stream), you'll use the Scanner object as:

   ```
   Scanner s = new Scanner(System.in);

   return s.nextInt();
   ```

   The Scanner class has to be resolved. Click on the word Scanner and press Ctrl-1 and choose *import Scanner*.

4. Create another method called **String getString**(**String** prompt)

   This method is similar to the **getInt()** method, except you should change the **s.nextInt** to `s.nextLine();`

5. Call both methods in the **main()** method and then print the result to test your code.



Part 2 – Performing data conversions

Step by step

The scenario is going to mimic a serving line at a lunch hall in that we are going to prompt the user to answer certain questions. What would you like as a main dish? How many roast potatoes? How many Brussels sprouts? Then, display what their lunch is.

1. Create a method called **`theLunchQueue`**. In the Program class.
2. Call the **getString**() method to display the following
   `What main dish would you like (fish, burgers, or veg)?`

   And get the answer into a variable called **mainCourse**.
3. Use the **getInt**() method to display the following prompts and capture the values in suitable variable names.

   ```
   How many roast potatoes would you like?
   How many Brussels sprouts would you like?
   ```

   Display the description for producing a bill. Something like:
   Hello, your lunch is xx with yy roast potatoes and zz Brussels sprouts.

   Replacing *xx*, *yy* and *zz* with your actual values of course!
4. Test your code by calling **`theLunchQueue() method`** from main().

Part 3 - Weight Conversions

Step by step

1. Create a method as
   `void convertInputToStonesPounds(int pounds).`

   This method should

   a. Ask the user for a total weight in pounds in **main**() and pass the result to this method.

   b. Display the result (stones & pounds) in this method.

      Note: there are 14 pounds in a stone.
      Hint: Use division (/) and modulus (%)
2. Create another method as
   `void convertKgsToStonesPounds(int kg).`

   a. Ask the user for a weight in kilograms.

   b. Convert the weight and display it in stones and pounds

      Hint: 1 kilo = 2.20462 pounds
      Tip: convert the Kg to pounds and then call
      `convertInputToStonesPounds`(int kg)
3. Test your code at each stage.



Part 4 – Move your code to a separate class

Does every method have to be in the Program class? In this part, you'll create a new class and move all the code to that class.

Step by step

1. Create a new Class called **Lab3Exercises** without a main() method in the lab3 package.
2. Cut all the code outside of the main() method (Program class) and paste them inside the *Lab3Exercise* class.
3. Remove the **static** word from every method definition.
   We'll discuss static method at a later date. The only reason why every method was static was because main() is a **static** method() but we are now free of main()!
4. Back in the **main**() method, create an instance of ***Lab3Exercises*** class and use it to call the methods.

   Lab3Exercise myLab3 = **new Lab3Exercise**();
5. At the start of each method call (in main) add "**myLab3**"
   For example:
   instead of `theLunchQueue()` type `myLab3.theLunchQueue()`
6. Run the application to make sure everything works.

** End **

Lab 4: Java – Getting Started with Conditionals

Objective

The objectives of this Lab session are to practise using conditional expressions (if, else, and switch).

Overview

In this lab called 'Kids in a Candy Store', you'll practice using if else statements

Introduction to 'Kids in a Candy Store'

In this lab, you will prompt the user for the price of a bag of sweets, in pennies, also the amount of money they have, also in pennies, and then perform a calculation to work out how many bags of sweets they can afford. Then it will be possible to display a friendly message like:

"The price is 'x'p and you have 'y'p then you will be able to buy 'z' bags".

Step by step

1. Back in the **labs** project which you created in **Lab1**, add a new package called lab04.

   Please refer to lab1's instructions if you need help.

2. Add a new class called Program to the *lab04* package with a *main()* method.

3. Add a class called **Lab4** to this package (with no main method)

4. Transfer the code for **getInt()** which you wrote in the previous lab to the Lab4 class.

5. Create a method in Lab4 as:

   ```
   public void part1() {
   }
   ```

6. Create an instance of Lab4 in the main() and call the part1() method to get ready for the rest of this exercise.

   ```
   Lab4  lab4 = new Lab4();
   lab4.part1();
   ```

   From now, all your code will go in the part1() method.

7. Ask the user "Price of a bag please?" and hold the result in a variable.



8. Ask the user “How much money do you have?” and record it in a variable.
9. Calculate the number of bags they can afford and store in a variable called `numBag`.
10. Now, display a message in the format
    “If the price is ‘x’p, and you have ‘y’p, then you will be able to buy ‘z’ bags”.
11. Build and test your code using values of ‘20’ for price and ‘45’ for money and ensure that it says ‘2’ bags can be purchased.
12. Test your code again using values of ‘20’ for price and ‘15’ for money and ensure that it says ‘0’ bags can be purchased.

    The message looks odd, and it will look even stranger if the user enters negative values. It will also crash the program if the user enters zero as the price of a bag. We need to bulletproof your code.
13. If the price of a bag is a negative number, then display a suitable message and return from the part1() method.
14. Similarly, if the user has zero or negative amount of money then you should display a suitable message and return.
15. Is there any other way you can protect this application from users?
16. You should now test your code using the following test values.

Price	Money	Outcome	Error Message
-10	n/a		“Price must be ….”
0	n/a		“Price must be ….”
10	-5		“Money must be …”
10	0		“Come back with your pocket money”
10	30	…can afford 3 bags…	

17. You should now test your code using the above test script.
18. You have now avoided any divide by zero exceptions being thrown.

** End **

Lab 5: Java – More Conditionals

Objective

In this exercise, you'll practice using the logical AND (&&) and OR (||) in an if statement.

Part 1 – Using logical AND (&&) and OR(II) in an if statement

Step by step

1. Back in the **labs** project which you created in **Lab1**, add a new package called lab05.

   Please refer to Lab1's instructions if you need help.

2. Create a new class called *Program* in Lab05 package with a main() method.
3. Add a class called Lab5 to this package (with no main method)
4. Transfer the code for **getInt()** which you wrote in the previous lab to the Lab5 class.
5. Create a method in Lab5 called grades as:

   ```
   public void grades() {
   }
   ```

6. Create an instance of Lab5 in main() and call grades() to get ready for the rest of this exercise.

   ```
   Lab5  lab5 = new Lab5();
   lab5.grades();
   ```

7. From now all your code will go in the grades() method.
8. Use the getInt() method to input an exam mark.
9. The rules for calculating a grade is as follows:

   If the mark is less than 1 or greater than 100, display:
   "Error: marks must be between 1..100"

Less than 50		Fail
between 50..60	(inclusive)	Pass
between 61..70	(inclusive)	Merit
between 71..100	(inclusive)	Distinction



Part 2 – Evening Behaviours (Optional)

In this part, you'll display different recommendations depending on if it is Summer and/or early evening

Step by step

1. Create another method called **Part2().**

   ```
   public void Part2() {
   }
   ```

2. In the Part2() method, ask the user if it is Summer. If they answer "yes" then set a Boolean variable to true.

   Tip: use the getString() method to get a String.

3. You will then ask the user to indicate if it is early evening and record this fact in another Boolean variable.

   Tip: create a bool variable that takes the value of **true** if the user says "**yes**" and will be **false** otherwise.

4. Using **if, else** statement/s. produce the following logic table in code to produce the output indicated.

Season: Summer	**True**	**True**	**False**	**False**
Condition: Sunny Evening	**True**	**False**	**True**	**False**
Action: Take a shower	✓	✓	✓	✓
Action: Eat outside	✓			
Action: Eat inside		✓	✓	✓
Action: Do outdoors hobby	✓	✓		
Action: Take a walk			✓	
Action: Contact friends	✓	✓	✓	✓

** End **

Lab 6: Java – Loops

Objective

In this lab, you'll practice using various looping constructs.

Part 1 – Calculating the grades for 5 students

Step by step

1. Back in the **labs** project which you created in **Lab1**, add a new package called **lab06**.
2. Create a new class called ***Program*** in this package with a main() method.
3. Add a class called **Lab6** (with no main method)
4. Copy the code for **getInt()** which you wrote in Lab6.
5. Create a method in Lab6 called **part1**():

   ```
   public void part1() {
   }
   ```

6. Create an instance of Lab6 in the main() and call the grades() method to get ready for the rest of this exercise.

   ```
   Lab6  lab6 = new Lab6();
   lab6.part1();
   ```

7. From now, all your code will go in the part1() method.
   We will revisit the grades() method that you wrote in Lab5 but this time we will process many students rather than just one grade.
8. Copy the code for processing grades() to the Lab6 class.
9. Call grades() from the part1() method.
10. Create an array of 5 names called names at the start of the grades() method

    Tip: View slides for code.

11. Create an array of 5 integers called **marks** to hold the marks for our 5 students.
12. Create a loop (while or for) to:
    a. Get a student name and store it in the **names** array
    b. Get the grade for the student and store it in the **marks[]**
13. Having stored the names of the students and their grades, create another loop to display each name, the *grade*, *mark* and *grade* (pass/merit…)



Part 2 – How long does it take to double your money?

Assuming an initial investment of say £100, how many years does it take to grow to £200, given an interest rate of 5 percent?

Step by step

1. Create a new method In account() in the Lab6 class.
2. Create suitable variables to store the initial money, current money (at the end of each year), interest rate (5%), and years (to double the money).
3. Write code to calculate the number of years it takes to get £200.

   Tip: Use a while loop which stops when the current money == £200

Part 3 – Nested Loop Practice

Ensure you can code up nested loops, understanding the full sequence in which everything runs, and effectively use the outer and 'inner' loop variables together in a nested loop. In this part you'll produce a multiplication table.

Step by step

1. Create a method called `multiplicationTable() in the Lab6 class`.
2. We want you to produce this output on the console.
   1 2 3 4 5 6 7 8 9 10
   2 4 6 8 10 12 14 16 18 20
   3 6 9 12 15 18 21 24 27 30
   4 8 12 16 20 24 28 32 36 40
   5 10 15 20 25 30 35 40 45 50
   6 12 18 24 30 36 42 48 54 60
   7 14 21 28 35 42 49 56 63 70
   8 16 24 32 40 48 56 64 72 80
   9 18 27 36 45 54 63 72 81 90
   10 20 30 40 50 60 70 80 90 100

   Tip: Two nested for loops (count from 1..10) are best for this.

   Also, to print the product of two variables called row and col in 5 spaces, use a statement like:

   System.out.printf("%5d", col * row);

** End **

Lab 7: Java – Arrays

Objective

The objectives of this practical are to get fully up to speed with array definition, creating, filling, and manipulation and to fully understand when it is appropriate to use a 'foreach' loop as opposed to a 'for' loop to process the array.

Part 1

Step by step

1. Back in the **labs** project which you created in **Lab1**, add a new package called **lab07**.

   Please refer to lab1's instructions if you need help.

2. Add a class called **Program** with a main() method to this package.
3. Add a class called Lab7.
4. Add a method called start() to Lab7 class.
5. Call the start() method from main().

   Tip: refer to instructions in previous labs if you're not sure how to do this.

6. Declare an array of integers in the start method as:

   ```
   int[] numbers = { 1, 3, -5, 7, 0, 4, 6, 8 };
   ```

   You can put all your code in the start() method or (even better) create individual methods for each one of the following tasks and then call them from the start() method.

7. Task 1: write code to find the sum of every number in numbers
8. Task 2: Find the average of these numbers
9. Task 3: Find the minimum number in numbers
10. Task 4: Find the maximum number in numbers
11. Task 5: Find the index of number zero in numbers



Part 2

In this part, you'll implement the bubble sort algorithm.

Step by step

1. Create a method called sort() in Lab7 class.
   This method should accept a parameter of type int[ ]
2. Call the sort method from the start() method and pass numbers as parameter.
3. Implement Bubble sort.

Please don't visit the geeksforgeeks.org website, as this provides an example, complete with the code! Instead, take a look at
https://en.wikipedia.org/wiki/Bubble_sort
for an explanation of how a Bubble sort works before writing the code yourself.

** End **

Lab 8: Types I – Getting started

Objective

The primary objectives for this lab are to:

- create and use references.
- consolidate on passing by value and passing by reference.

Part 1 – Creating and using reference types

Step by step

1. Back in the **labs** project which you created in *Lab1*, add a new package called **lab08**.

   Please refer to lab1's instructions if you need help.

2. Create a new class called *Program* in this package with a main() method.
3. Create another class called Account with following private fields:
   ```
   int id;
   string owner;
   double balance,
   ```

   and methods:
   ```
   void Deposit(double amount) { }
   void Withdraw(double amount) { }
   ```

4. Make sure no one can deposit a negative amount of money!
5. Also make sure no one can withdraw money they don't have.
6. Create a constructor for the account class to set its state.
7. Create a method called **getDetails**() to return details of the account to console.
8. In the `main()` method, create an instance of the account class.
9. Invoke its deposit() and withdraw() method.
10. Call the account's getDetails() method and print the result to make sure your code has had the expected effect.



Part 2 – Passing reference types to a method

In this part, you'll examine passing reference types between methods. This is an important topic which has security implications.

Step by step

1. In the Account class, create a new method as:
   public void **AddInterest**()

   Write code in this method to add 2.5% interest to the balance.

2. Add code in main() to create an account called ***myAccount***,
   with a balance of £100.

3. Call the addInterest method of the myAccount instance.

4. After the call, display the details of myAccount
   (using its getDetails()).
   Did the balance change?

5. Create another Account reference called partnerAccount and set it to the myAccount instance as:
   Account partnerAccount = myAccount;

6. Call the partnerAccount.addInterest() method

7. Call myAccount.getDetails() method. Did the balance change?

Let's do another experiment

8. Create a method in the **Program** class (just under the main())
   static void **processAccount**(**Account acc**){
   acc.addInterest();
   }

9. Back in Main(), create an instance of Account called *myAccount* and call the processAccount method, passing myAccount as parameter.

10. Call myAccount.getDetails() method. Did the balance change?

11. Let's do another important experiment.

12. Create another method under the main() method as:

```
private static void incInt(int x) {
        x++;
}
```

13. Add code to the main() method to create an integer like
    int k=100

14. Call incInt and pass it k

15. After the call, print the value of k.

    Did k change after the call?



All primitive data types (built into the language) are **Passed by Copy** (also known as **passed by value**). Therefore, changing x did not change k and changing the name of k to x will not alter anything!

All Reference types (like myAccount) are also passed by copy but what is passed is their address in memory. Therefore, when you call a method like **processAccount**(**Account acc**) and pass it an account reference (such as myAccount) you are actually passing its address! The **acc** reference will be pointing to (referencing) myAccount. Any change made by **acc** will directly affect myAccount.

** End **



Lab 9: Types II – Fields and Methods

Objective

You will further your understanding of object-oriented programming.

In this lab, you'll create a graphical application instead of a Console app.

Part 1

You'll soon be creating a graphical application which is new. You'll copy and paste many lines of code. You are not required to understand how the graphics code works but it is not too complex to investigate by yourself.

Step by step

1. Back in the **labs** project which you created in *Lab1*, add a new package called **lab09**.

   Please refer to lab1's instructions if you need help.

2. Create a class called **Game** in this package.
3. Create an instance of Game in the **main()** method. This will kick start the game using its constructor.
4. You need a canvas upon which you can draw and show your animation. In this case you'll create a number of ball objects bouncing around a rectangle (the world).

   You can get creative later and do something else but for now please follow the steps. The aim is to investigate OO and not to concentrate on animation and graphics.

   Delete any code in the **Game** class and copy the following code.

   There a few comments in this code which are placeholders for code which you'll write later on.

```
import java.awt.*;
import java.awt.event.*;
import java.util.*;
import java.util.Timer;
import javax.swing.*;

public class Game extends Canvas {
	// create an array of 3 balls
	// call this array balls

Game() {
		JFrame frame = new JFrame();
		this.setSize(400, 400);
		frame.add(this);
		frame.pack();
		frame.setVisible(true);

		Timer t = new Timer();
		TimerTask tt = new TimerTask() {
			@Override
			public void run() {
				draw();
			}
		};

		t.schedule(tt, 0, 50);


		frame.addWindowListener(new WindowAdapter() {
			@Override
			public void windowClosing(WindowEvent e) {
				t.cancel();
				tt.cancel();
			}
		});
	}

	public void draw() {
		// call the move() method of each balls
		// Tip: use an enhanced for loop to pick
		//  each ball in the balls array.
		this.repaint();
	}

	public void paint(Graphics g) {
		g.drawRect(0, 0, 300, 300);
		// move and draw each ball in balls array
		// Tip: use an enhanced for loop to pick
		//  each ball in the balls array.

	}
}
```

Creates a form

Creates a timer to call the draw() method every 50ms (1/20 sec)

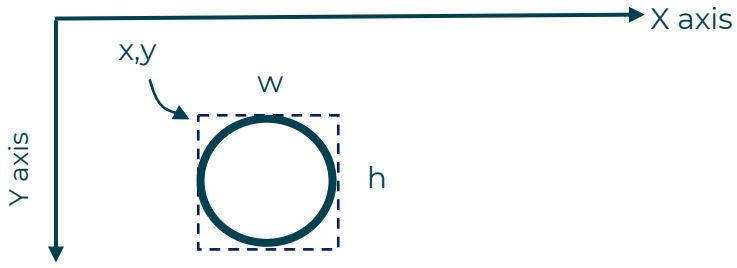
Runs when the window is closed

Invokes a repaint which will run the paint() method

5. Run the application to see if your code works as expected.

6. Create a class called **Ball** with the following fields

```
public int x, y, w, h;
private int dirX, dirY;
```



A ball's dimension is represented as a rectangle. X and y are the top coordinate and w and h are the width and height of the ball's rectangle. dirX and dirY are the amount by which each ball moves in the x and y direction.

The coordinate system is perhaps unlike what you're familiar with as it is upside-down! X increases to the right and Y increases towards the bottom.

7. Create a constructor to set every field (x,y,w,h,dirX,dirY).

8. To practice using constructor chaining, create another constructor to set just x,y,w,h but not the dirX and dirY.

   Use constructor chaining to call the first constructor with dirX=1 and dirY=1.

9. Create a method called **move()** in the **Ball** class.
   In this method, increase x by dirX and y by dirY and then follow a series of tests to make sure each ball stays within the world which is set in the **paint** event as 300px wide, 300px high and starts at x=0,y=0.

   if (x < 0) then set x=0 and change the sign of dirX
   Tip: **dirX = -dirX**
   do the same for the y coordinate (when it goes below zero) but this time changing dirY.

   if (x > 300-width of the ball) then set
   x=300-width of the ball and reverse the sign of dirY.

   do the same for the y coordinate (taking the height of the ball into account).

10. Back in the Game class view the comments and perform those tasks (create 3 balls, call move()... and draw each ball).
    Tip: paint method can draw a circle using this code:
    g.drawOval(b.x, b.y, b.w, b.h);
    // where b is a ball reference

11. If everything works, you should see 3 objects bouncing on your screen. If you have a need for speed then increase the dirX and dirY values!



Part 2

In this part, you'll make use of static keyword.

Clearly setting the world dimension to 300px (height and width) is not a good idea.. All balls share the same world coordinates. This is a perfect place to use the **static** keyword.

Step by step

1. Create a static int field in Ball called **worldW** and another called **worldH** to represent the width and height of the world. You should see three balls bouncing!
2. Create static method as
   public static void setWorld(int w, int h)
   to set the static *worldW* and *worldH* values.
3. In the Game() constructor just before the line `Timer t = new Timer();`
   Invoke the setWorld() method of the Ball class to set the world's width and height for every Ball.
4. Change the move() method in the Ball class to use the Ball.worldH and Ball.worldW values instead of 300px. You'll also need to change the paint() method to use the new static values (to draw the world's rectangle).
5. Run and test your code.

** End **

Lab 10: Enums and Strings

Objective

See how to define and use the `enum` keyword to define a new type. Consolidate on knowledge of the functionality of class `String` and introduce the very useful `StringBuilder` class.

Part 1 - Using an enum

Step by step

1. Open the Game project you created in Lab 9.
2. Circle is not the only shape! You can give the Ball class a property that dictates its shape. However, the shape must be limited to a list which you define. You will need to create this type as an **enum**.
3. Define a new enum called SHAPE_TYPE with the following values

   Rectangle
   ThreeDRectangle
   RoundRectangle
   Oval
   Arc

   Please create this *enum* outside of the Ball class or in its own file.
4. Now you can see the name **Ball** does not look like a good choice!
   Please change the name Ball to **Shape** in your project. The best way to do this is to open the Ball class and right mouse click on the word Ball and then choose the **Refactor->Rename** menu options. The editor will change all references to Ball.
5. Define a new private field called shapeType of type SHAPE_TYPE
   as: **private SHAPE_TYPE shapeType;**
6. Create a getter for this field.
7. Set this value inside the constructor.
   Tip: Add a parameter of type SHAPE_TYPE to the constructor.
8. Back in the paint method, you can now examine the getShapeType() to see what to draw. for example

   ```
   if ( getShapeType() == SHAPE_TYPE.RoundRectangle)
                              g.draw drawRoundRectangle(.....);
   ```
9. Run your application to see different shapes bouncing about!
   You can also change the colour of your shape by creating a new field of type Color (like: private Color colour;)
   Set its value in the Shape's constructor and also create a getter method.
   You can then use this in paint when drawing a shape using code like:
   `g.setColor( shape.getColour() );`
   `g.drawRect(...);`



Part 2 - Using String

Step by step

1. Expand `main()`, declare a `String` called name whose value is any first name of any length greater than 3 characters.
2. Display its 3rd character using `charAt()` (can also be done with substring)
3. Display it converted to lowercase and to uppercase.
4. Use an enhanced `for` loop to iterate over its characters (use `toCharArray()`)and display each of them tab separated. Throw a line feed after this display.
5. Display whether it `startsWith` a `String` of your choosing.
6. Display whether it `endsWith` a `String` of your choosing.
7. Use `indexOf` to display the position in the `String` of the first occurrence of a character that you know is in the `String`, and also for a character that you know is not in the `String`.
8. Concatenate the 'name' with a surname of your choice to make a variable called '`fullname`' preferably with a space in the middle, then display this `fullname`. Concatenation is ok if it is all done in one statement.

Part 3 - Using StringBuilder

Step by step

1. Back in main(), create a `StringBuilder` object called '**sb**'. Use the constructor that allows you to initialise the object to contain the `String`

   "Bruce Springsteen<space>".

   (You can use the name of your favourite artist instead!)
2. Now use the `append()` instance method of *`StringBuilder`* to append exactly the text "is the artist ever" (no error in that!).
3. Use the `toString()` method of the `StringBuilder` to produce a string that you can display to see the current value of the `StringBuilder`.

   You are looking at a strange sentence that needs some amending.
4. Now we would like you to `insert()` an adjective in front of the word "artist". Words like "greatest" obviously spring to mind but make your own choice.
5. Now use the `replace()` method of `StringBuilder` to replace the word

   "artist" with a noun of your own choice. e.g., "rock singer". Display the final result.

** End **



Lab 11: Collections and Generics

Objective

In this lab you will practice using ArrayList, Queues and in a separate task, you'll work with a HashMap.

Part 1 – Use an ArrayList<Shape>

Step by step

1. Open the bouncing shape exercise that you did in the previous lab.
2. Change the array of Shapes (in Game class) to ArrayList<Shape>.
   Tip: you'll need to use the ArrayList's add method in the constructor to add the three shapes.

Part 2 - Using a Queue and a Stack

Step by step

1. Back in the **labs** project which you created in *Lab1*, add a new package called **lab11**.
2. Add a new class called *Program* to this package with a main() method.
3. Add another class called ShoppingBasket with following fields
   String productName.
   int quantity.
   double price
4. Create a constructor for the ShoppingBasket class and then create a method called displayDetails() to display the fields' values in Console.
5. In the Program class, create a Queue of ShoppingBasket called **baskets**. Since we are going to use this queue in main(), it needs to be static.
6. Create a static method in the Program class called buy().
   Call the buy() method from within main().
7. Write code in the buy() method to add a few **ShoppingBasket**s to the **baskets** Queue. We will process these in another method.
8. Create another static method called **processBaskets**()
   Write code to remove items from the **baskets** queue and call their **displayDetails**() method.

   This method simulates processing of shopping baskets for payment and shipping. For simplicity we just investigate the queue actions.
9. Call the processBaskets() method in main().
10. Run and test your code.



Part 3 - Using HashMap<K,V>

Scenario: A Zoo has a number of current animals and is expecting new arrivals soon. They wish to keep track of which animal types they have and record the count of each animal type.

You will create HashMap<String, int>. The key of String to store animal type and a value of Integer for the count of occurrences (Lion 3, Zebra 2 etc).

Step by step

1. Comment out the code in **main()** to get ready for this exercise.
2. Create a class called Zoo with a default constructor.
3. Create an instance of Zoo in main(). This will kick-start the constructor where all your code will be placed.
4. Declare and initialise the following fields.

   The following String arrays contain the names of the existing animals and the new animals we will add to our zoo. The HashMap will keep track of the animals and their count.

   HashMap<String, Integer> ***animalMap*** = **null**;
   String[] ***originalAnimals*** = {"Zebra", "Lion", "Buffalo"};
   String[] ***newAnimals*** = {"Zebra", "Gazelle", "Buffalo", "Zebra"};
5. Instantiate the animalMap (new it) in the class constructor.
6. Create a method in Zoo as **void** addAnimals**(String[ ]** animals**)**
7. Call `addAnimals()` twice from the constructor and pass each of the 2 arrays (***originalAnimals*** *and* ***newAnimals***) to this method**.**
8. **addAnimals**() should iterates over the `String[]` (method parameter) and add each String to the animalMap. If the animal is in the HashMap then its count will be increased by 1 otherwise its count will be set to 1.

   Tip: use HashMap's `containsKey()` to see if animal exists in the map.

   You will also use the HashMap's `put()` method to put an entry back in the collection and use the HashMap's `get()` method to get the count of an animal type.
9. Create a new method called `displayAnimalData`(). Call this method from the constructor.

   Tip: Best use and enhanced for loop.

   This method is going to display key/value pairs in 2 columns like this.
   Zebra 3
   Lion 2
   Buffalo 5
10. Run and test your code.

** End **



Lab 12: Java - Working with Files

Objective

The objective of this practical session is for you to become familiar with a number of classes found in the `java.io` namespace and practice reading and writing to files.

Part 1 – File I/O Practice

Step by step

1. Back in the **labs** project which you created in *Lab1*, add a new package called **lab12**.
2. Add a new class called **Program** to this package with a *main*() method.
3. Add three pre-supplied text files to your ***src*** folder.
   course.txt, trainer.txt and courseTrainer.txt

   Tip: to add these, right-click on the src folder and then select **import** > **general** > **file system**.
4. Create a method in the Program class with the following signature

   private static String[] readCSV(String inFile) **throws** IOException {
   }

   We've not covered Exception handling so just ignore the **throws** IOException part for now!

   Please also change the main() as:
   public static void main(String[] args) **throws** Exception {

5. Write code in the readCSV method to read one line the inFile parameter and split the resulting String using "," as delimiter. Return the String[ ] returned from the split method.

   Tip: use the String's split(",") method. This method returns an String[ ].
6. Back in main(), call the readCSV() method and pass it the "src/course.txt" as file name and capture the result back in a variable called **courses**.

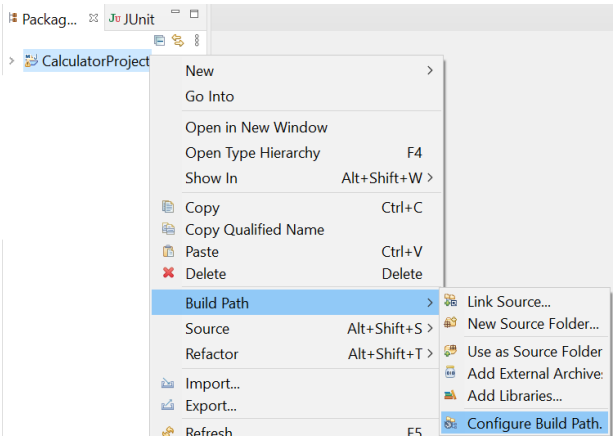
   Do the same for "src/trainer.txt" and capture the result in a variable called trainers.
7. Create a file called **trainersAndCourses.txt** and append the courses and trainers arrays to this file.
8. Run your code and make sure it works.

Part 2 – Reading JSON files

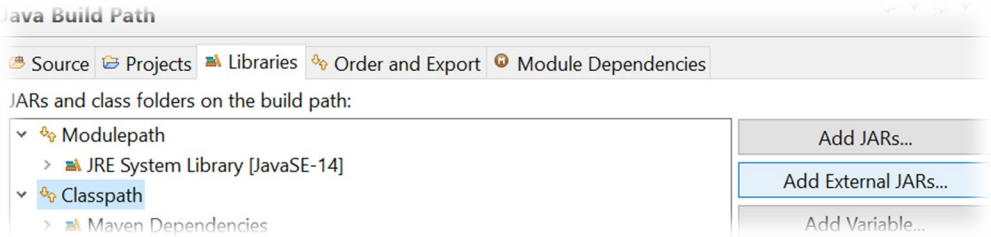
Step by step

1. In this part of this Lab, you'll create some code to read a JSON file and parse the text in order to extract an array of customers stored within it. Create a new project (or add a class to an existing Java project) to read and process JSON data.

2. Before you can use the Gson module, you need to import it. Please follow these steps.
   Right-mouse click on your project and then select the "Build Path" > "Configure Build" Path options



3. Select the "Class path" option and then click the "Add External JARs..." button. Select the provided "gson-2.6.2.jar" file in the Windows Explorer. Click the "Apply and Close" button to complete the import.



4. View the provided "customers.json" file in an online JSON viewer (such as http://jsonviewer.stack.hu/) and you'll see that it contains an array of customer object.





You'll soon read the customer data and create an array of customers. Therefore, you'll need to create a customer class

```
class Customer {
      String CustomerID, CompanyName, ContactName, ContactTitle;
      String Address, City, Region, PostalCode, Country;
}
```

You would usually declare these fields as `private` and then use your IDE to create getters and setters for the private fields, but we'll leave these as internal to the package to concentrate on the main subject.

5. Please copy the following code to your main() method or create another method to house it.

```
public static void main(String[] args) throws Exception{
     Gson gson = new Gson();
    String fileName = "C:\\qa\\customers.json";
     String content = new String(Files.readAllBytes(Paths.get(fileName)));
    Customer[] customers = gson.fromJson(content, Customer[].class);
    System.out.println(customers[0].CustomerID);
}
```

The above code prints the first customer's **CustomerID**.
Please note, the spelling of this field is not accidental but matches the text in the **customers.json** file.Please create a loop to print more of the data.

6. Run the code to test it.

Writing objects as JSON:

7. How would you write a single object to a file?
   How about a List of objects?
   Here is a sample code to get you started:

```
Gson gson = new Gson();

int[] nos = {1,3,5,7,9};
String res = gson.toJson(nos);
System.out.println(res);

String[] names= {"Bob","Fred","Wilma"};
res = gson.toJson(names);
System.out.println(res);

Student[] students = {
   new Student("Bob",21),
   new Student("Fred",32),
   new Student("Wilma",26)
};
res = gson.toJson(students);
System.out.println(res);

//----------------------------------
class Student{
   String name;
   int age;
   ArrayList<String> subjects = new ArrayList<>();
```

```
    public Student(String name, int age) {
      this.name = name; this.age = age;
      subjects.add("Java");
      subjects.add("C#");
      subjects.add("Network");
    }
}
```

Part 3 – Relate courses and trainers (optional and only if you've time)

Step by step

8. Add a pre-supplied text file called **trainerCourse.txt** to the **src** folder. This file contains rows of two **","** separated integers.
9. The two integers point to trainer numbers (the first value) and course titles (second number).
10. Trainer and course numbers being the order in which trainers and courses are stored in the files.
11. For example, **`1,2`** indicates
    "Mike Baradaran" (**1st** row of trainer.txt), teaching
    "Systems Development Basics" (**2nd** row of course.txt).
12. Convert all the numbers to trainer names and course titles in order to produce a report.

** End **



Lab 13: Exceptions

Objective

The primary objectives for this lab are to be able to throw and catch exceptions and have a full understanding of the Java runtime's propagation mechanism for exceptions until a handler is found.

Overview

Firstly, you will handle the exceptions by throwing an exception object in your Account class if balance goes into red.

In the second part of this lab, you'll handle Java's Checked exceptions.

Part 1 - Using try/throw/catch in the Account class

Step by step

1. Add a new package to the **Labs** project called **lab13**.
2. Create a class in this package called **Program** with a *main()* method.
3. Create a class called Account with the following fields and methods
   a. int **id**; double **balance**; String **owner**
   b. Create a constructor to set these fields
   c. void withdraw(double amount) // **reduces the balance by amount**
   d. void deposit(double amount) // **increases the balance by amount**
   e. void **close**()
      just display a message like "account 123 is closed" where 123 is the Account's **id**.
   f. String **getDetails**() to get details of id,balance and owner.
   g. add suitable code to the withdraw, deposit and getDetails() methods
4. Back in the main() method create an account with balance of £100.
5. Withdraw £50 and then display the account using the getDetails() method.
6. Withdraw £60 and display the account's details
   Please note, there is no withdrawal limit. Such an excellent account!
7. Fix the withdrawal() method so that it throws an `IllegalArgumentException` when the balance is negative,
8. Catch this exception in main() and display a suitable user-friendly message.
9. Build and run the program now. Make sure you get the Account class to



throw the exception and make sure it is handled in main().

10. Add a **finally** clause to the exception handler in main() to close the account no matter if there is an exception or not.

Part 2 – Exceptions

Step by step

11. Create a method in the Program class to write a message to a log file.
12. Type the following code:

```
private static void log(String msg){
        File file = new File("log.txt");
        FileWriter fr = new FileWriter(file, true);
        BufferedWriter br = new BufferedWriter(fr);
        br.write(msg + "\r\n");
        br.close();
        fr.close();
}
```

13. Call the log method from main()
14. Your code will not compile unless you change the signature of the log() method. It should indicate that it throws IOException
15. Catch this exception in main() and display a IOException's message, should the log() method fail.

** End **

Lab 14: Inheritance – Getting started

Objective

The primary objective for this lab is to enable you to derive new types and add specialist functionality.

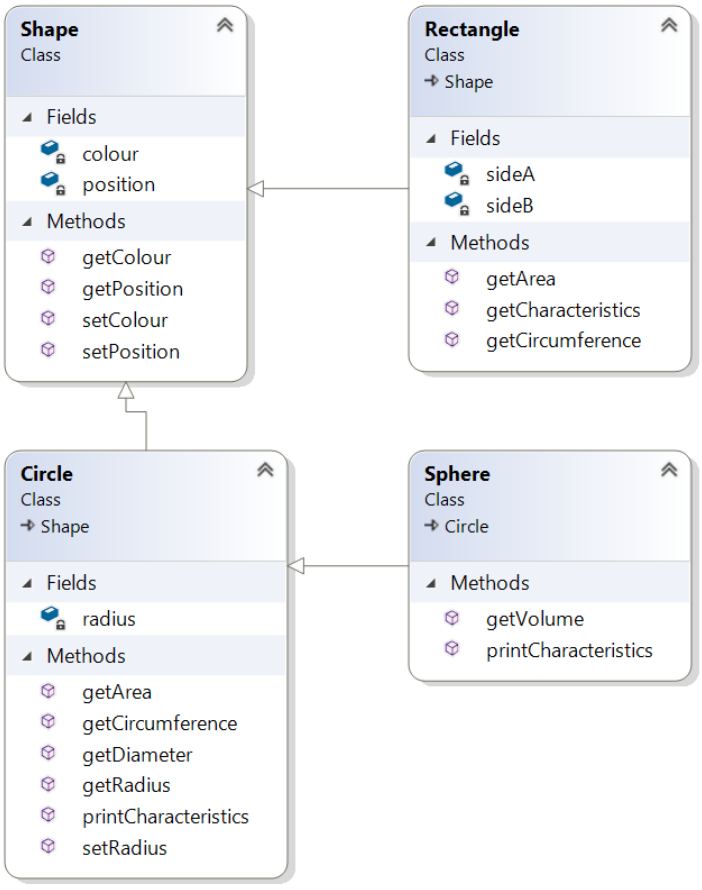
Overview

The lab introduces some of the basic concepts of the inheritance story. As mentioned in the associated session, in order to implement inheritance, you must first have a class that provides the fundamental definition or behaviour you need.

In this lab we will play about with circular shapes.

Step by step

1. Open the **Labs** project and then add a new package called **lab14**.
2. Create a new class called Program with a main() method in the lab14 package.
3. Create the following class structure





4. Create a constructor for Shape to set its colour and position

5. As you can see, **Circle** extends **Shape** and **Sphere** extends **Circle.**
   a. ***Position*** is of type **Point** which is a class with built-in x and y.
   b. ***Colour*** is of type **java.awt.Color**

   Tip: Use **Math.PI** to get the value of **PI**.
   You will need this to calculate the area and circumfrance of circle.

   Volume of a sphere is calculated as 4/3 * PI * R^3 (R to power of 3)
   You can use the Math.pow() fuction or R * R * R

6. Create getters and setters for each of the fields (colour, radius...) as indicated in the class diagram above.
7. The **getCharacteristics()** method returns a ***String*** containing all the attributes of the shape. It will be up to the caller how to display this information.
8. Create a few shape types in main().
9. Print the characteristics of the Rectangle, Circle and Sphere objects which you've created.
10. Create an ArrayList<Shape> called **shapes** in the main()

11. Add the shapes which you created earlier into the *shapes* ArrayList.
12. Create an enhanced **for** loop to scroll through each shape and print its colour and position (x,y).

** End **



Lab 15: Inheritance – towards polymorphism

Objective

The primary objective for this lab is to enable you to derive new types and to override functionality.

Overview

Read the instructions below and critically evaluate each code sample. Trust nothing and make comments that will be reviewed as a class.

Part 1 – Racing Cars

Scenario

You are going to work with a project which consists of a `Car` class, a `RacingCar` class and a `Program (test)` class that fills a `Car[]` array with Cars and RacingCars objects.

The `Program` class will then process each `Car,` setting their initial speeds to 60MPH (this is done using a blatant cheat that normal cars don't usually possess – a `getToSixty()` method). The test will then continue by making each '`Car`' `accelerate` for 2 seconds before writing their `model` and `speed` to the `Console`.

If the car is a `RacingCar` we will need to add some additional code to the `Program` class so that a little extra information is written out.

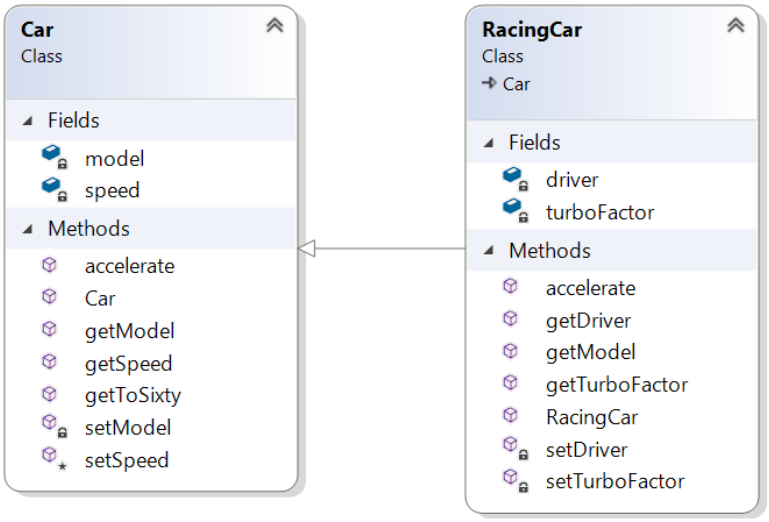
Step by step

1. Open the Labs project.
2. Create a new package called **lab15**
3. Add a class called **Program** with a **main()** method to the lab15 package.
4. Add a class called Car to the new package.
5. Add another class called RacingCar. This class extends Car.
6. The Car and RacingCar classes will have the fields and methods as displayed in the class diagram below.

7. You will also need to add constructors for the two classes.

   Please study the diagram below before writing code.

   Please also note that any method shown below with the same name as the class is the constructor for that class.



8. Add code to the `Car` class.

   Create getters and setters for the **model** and **speed** fields.
   The **getToSixty()** method just sets the **speed** field to 60.

   The accelerate method should look like **void** **accelerate**(**int** *seconds*) and increase the speed by *5 * seconds*.

9. Create a suitable constructor for the Car class.

10. Add extra code to the RacingCar class.

    Add a **String** driver field with getters and setters.
    Add an **int** turboFactor with getters and setters.

11. You'll have to create a suitable constructor for the RacingCar class.

12. The **accelerate()** method will invoke the base class (Car) accelerate method and then multiply the **speed** by the **turboFactor**.

    Tip: use the **super**.**accelerate()** method.

13. The **main()** method should create an array of Cars comprising of a few cars and racing cars.

14. The main() method should then pass the cars array to a method called `processCars`.

    The `processCars` method currently:

    - Gets each '`Car`' up to 60MPH as a start point.
    - Then accelerates each `Car` for 2 seconds.

- And displays details of each car.
- You'll also display the driver's name but only if a `Car` in the array is a `RacingCar`.
    - You'll need to examine the type of car in the array and if it is a RacingCar, cast it to a RacingCar in order to get the driver's name.

      Tip: use **instanceof** like: if (**c** **instanceof** *RacingCar*) {....}
      where '**c**' is a car element in the array.

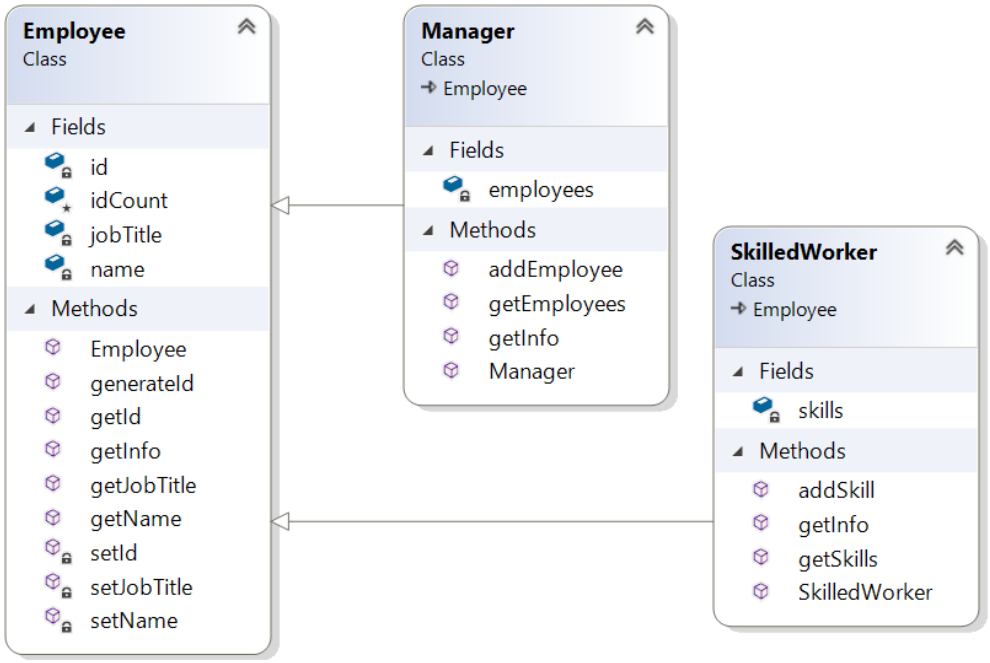
Part 2 – Employee Hierarchy

Scenario

You will design and create a class hierarchy for an employee tracking system.

Step by step – Designing the Hierarchy

15. Create the following classes. The employee class is provided below. Just copy and paste. We'll need you to concentrate on the other two classes.



16. The Manager class holds an **ArrayList**<Employee> called employees.
17. The **addEmployee(Employee emp)** method adds the **emp** object to the **employees** ArrayList.
18. The **getInfo()** method of the Manager should first gather the manager's details (using super.getInfo()) and then use a for-loop to go through the **employees** ArrayList in order to call their getInfo() method.
    It should then return the resulting string.
19. The **SkilledWorker** has an **ArrayList** <String> called **skills** which holds the names of skills possessed by a SkilledWorker instance.



20. The **addSkill(String skill)** method adds the **skill** String to **skills**.

21. In the main() method
    1. Create a Manager instance.
    2. Create a few regular Employee instances and add them to the manager's employees ArrayList
    3. Create a SkilledWorker object with a few skills and then add the instance to the Manager's employee ArrayList.
    4. Call the manager's getInfo() method and display the result.

    Question: Can we add a manager instance to a manager's employees?

To get you started here is the code for the Employee class:

```
public class Employee {

	private String name;
	private String jobTitle;
	private int id;
	protected static int idCount;

	public String getName() {  return name; }
	private void setName(String name) { this.name = name; }

	public String getJobTitle() { return jobTitle; }
	private void setJobTitle(String jobTitle) { this.jobTitle = jobTitle;}

	private void setId(int id) { this.id = id; }
	public int getId() { return id;}

	public Employee(String name, String jobTitle ) {
		setId(++Employee.idCount * 10);
		setName(name);
		setJobTitle(jobTitle);
	}

	public String getInfo() {
		String info = "\n**** *****";
		info += "Name: " + getName() + "\n";
		info += "Job Title: "+ getJobTitle() + "\n";
		info += "Employee ID: "+ getId()+ "\n";
		return info;
	}
}
```



Lab 16: Pair Programming

Objective

The primary objective for this lab is to work in pairs, using one of the pair programming models you learnt about in the Discover stage of your Programming with Java Learning Path - for example, driver-navigator, backseat navigator or tour guide. Make sure you swap roles at least once during the lab. If you are unsure about these concepts, ask your tutor for a reminder.

Part 1 – Factorials

Most people are familiar with the factorial operator in math. For example:

- 5! = 120

This is because factorial means "multiply successive terms where each are one less than the previous. So:

- 5! = 5 * 4 * 3 * 2 * 1 = 120

Now let's reverse it. Write a function that tells us that:

- 120 = 5!

Hint: divide the term by successively larger terms until you get to "1" as the resultant:

- 120 -> 120/2 -> 60/3 -> 20/4 -> 5/5 -> 1 => 5!

Sample Input/Output: *Your program should report what each number is as a factorial, or "NONE" if it's not legitimately a factorial. For example:*

- *120 => 5!*
- *150 => NONE*

Challenge Input: *These are the inputs you should use for this exercise. The results should either be the factorial, or "NONE":*

- *3628800*
- *479001600*
- *6*
- *18*



Part 2 – Counting Words

In this task, you must write a simple Java program that counts the numbers of times each word appears in a sentence.

Instructions:

- Take user input for a sentence.
- Implement a function that counts the frequency of each word in the sentence.
- Display the word frequency count to the user.
- The way you choose for format the result is not important.
- It does not need to be case-sensitive.

Sample Input/Output:

Input: “They opened the door, closed it, and then opened it again."

Output:

They = 1

Opened = 2

The = 1

Door = 1

Closed = 1

It = 2

And = 1

Then = 1

Again = 1

Challenge Input: *These are the inputs you should use for this exercise:*

- *“The cat chased a mouse. The mouse ran and the cat chased it again“*
- *"I eat ice cream. Ice cream is my favourite, and I eat it every day."*
- *“My car is red, because red is the best colour for a car”*

End

Lab 17: Unit Testing

Exercise 1 – Testing existing code

This lab uses the **Calculator** class found in the **exercise1** package of this repository. Clone the repository and import the project into Eclipse to get started.

Exercise: MrWalshyType2/QAA-Module3-UnitTest-Exercises (github.com)

Although you are advised to attempt writing the code by yourself first – If you don't have time to complete all of this exercise, here is a link to the solution for reference.

Solution: QAA-Module3-UnitTest-Exercise-Solutions/CalculatorTest.java at main · MrWalshyType2/QAA-Module3-UnitTest-Exercise-Solutions (github.com)

a) Create a test plan

In this exercise, you are required to create a test plan which consists of test cases for the **Calculator** classes 4 methods. Use the following template for creating your test cases:

ID	Method	Description	Inputs	Expected output	Actual output
1	add(double num1, double num2)	Adding two small numbers	num1=10 num2=30	40	



One test case has been created for you as an example. It is expected that you produce at least 3 test cases per method:

- Test borderline input values, i.e., what are the highest values you can add, what about the smallest...
- Test at least 1 normal input combination

b) Implement your test plan

The **Calculator** class has already been created, use your test plan to guide the development of tests for the methods of this class.



Exercise 2 – Testing exceptions

This exercise uses the **UserService** class defined in the **exercise2** package of the repository. Clone the repository and import the project into Eclipse to get started.

Repository: MrWalshyType2/QAA-Module3-UnitTest-Exercises (github.com)

Although you are advised to attempt writing the code by yourself first – If you don't have time to complete all of this exercise, here is a link to the solution for reference.

Solution: QAA-Module3-UnitTest-Exercise-Solutions/UserServiceTest.java at main · MrWalshyType2/QAA-Module3-UnitTest-Exercise-Solutions (github.com)

a) Create a test plan

In this exercise, you are required to create a test plan which consists of test cases for the **UserService** classes 2 methods. Use the following template for creating your test cases:

ID	Method	Description	Inputs		Expected output	Actual output
1	login(String username, String password)	Register a valid user, login successfully with said valid user.	login	username="bobby" password="Codes123"	"bobby"	
			register	username="bobby" password="Codes123"		
2	register(Strin gusername, String password)	Register a user with an invalid password due to missing number.	register	username="bobby" password="Codes"	IllegalArgu ment Exception ("Passwor d must contain at least 1 number character")	

Two example test cases have been created for you. It is expected that you produce a test case for every possible exception that could be thrown.



b) Implement your test plan

The **UserService** class has already been created, use your test plan to guide the development of tests for the methods of this class.

Exercise 3 – Mocking in a unit test

This exercise relies on the **User**, **UserController** classes and **UserRepository** interface in the **exercise3** package in the supplied repository.

Repository: MrWalshyType2/QAA-Module3-UnitTest-Exercises (github.com)

a) Update the test plan from Exercise 2

The test plan from Exercise 2 can be reused for this example. Modify the test plan to accommodate the changes to the **login** and **register** methods present in the **UserController** class.

- As we are now dealing with multiple classes, it is also recommended to add a **Class** column to the test table.
- There is a new exception that could be thrown in the **register** method.
- Some exceptions have been removed from the **login** method as it is expected that the repository implementation would handle those cases in this example, i.e., invalid usernames or passwords.

b) Implement the tests

Implement your unit test plan, as done with the previous examples.

- Be careful when writing your tests for the **login** and **register** methods, it is expected that you use the Mockito framework to mock interactions with the repository.
- Make sure to mock the repository and inject it into the controller with the @Mock and @InjectMocks annotations respectively.
- The **repository** methods being mocked are: **UserRepository.exists()**, **UserRepository.register()** and **UserRepository.login()**

c) Test driven development (Stretch task)

If you complete the above task, create a test plan for the methods of the **UserRepository** interface.

Once a suitable plan is created, create your tests, and implement the interface as a class, call it **ConcreteUserRepository**.



- The **Concrete** in the name indicates that it is a class and not an interface or abstract class.
- Store the instances of **User** in a **List<User>** instance variable on the concrete repository class.

Advice

1. After creating the plan, create the concrete repository class and implement the **UserRepository** interface.
2. Add the empty method stubs.
3. Create the test class **UserRepositoryTest**.
4. Start creating the **register** tests.
5. Create the implementation of **ConcreteUserRepository.register()** as you write the test.
6. Repeat steps 4 and 5 for the **login** and **exists** methods.

** End **



Lab 18: JDBC

Objective

In this lab, you will

- create a new database using MySQL.
- create a new table using MySQL.
- write a program to perform CRUD based operations on your local MySQL server, using the database and table created in the previous steps.

Step by step

1. Create a new Database in MySQL. You can do this using MySQL Workbench.
2. Create a new table in your new Database, this can be anything, but try to have at least two columns. Again, Workbench GUI can be used here instead of a create table statement.
3. Create a new maven project in Eclipse.
4. Add the mysql connector dependency listed in the slides (or from mvnrepository.com) to your pom.xml
5. Follow the tutorial in the slides and make sure your program can:
    - Create
    - Read
    - Update
    - Delete

Stretch Goal

Use preparedStatement wherever possible, rather than concatenated variables.

End

